

1. **You are designing a system where different types of vehicles have their own way of starting. Create a parent class Vehicle with a method start(). Create subclasses Car and Bike that override the start() method. Write a program to demonstrate dynamic method dispatch when calling start() using a Vehicle reference.**

Code:

```
class Vehicle{
    void start(){}
}
class Car extends Vehicle {
    void start() {
        System.out.println("car: car car car");}
}
class Bike extends Vehicle {
    void start() {
        System.out.println("bike: bike bike bkie");}
}
public class One {
    public static void main(String[] args){
        Vehicle sakib=new Car();
        Vehicle adid=new Bike();
        sakib.start();
        adid.start();}
}
```

2. **Suppose you are creating a simulation of animal sounds. Define a class Animal with a method makeSound(). Create subclasses Dog and Cat that override makeSound(). Demonstrate dynamic method dispatch by calling makeSound() using an Animal reference.**

Code:

```
class Animal{
    void makeSound(){}
}
class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog: bark bark bark");}
}
class Cat extends Animal {
    void makeSound() {
        System.out.println("Cat: meow meow meow");}
}
public class Two {
    public static void main(String[] args){
        Animal sakib=new Dog();
        Animal adid=new Cat();
        sakib.makeSound();
        adid.makeSound();}
}
```

- 3. You are developing an e-commerce payment system. Create a class Payment with a method pay(). Create subclasses CreditCardPayment and PayPalPayment that override pay(). In the main() method, use a Payment reference to demonstrate runtime polymorphism.**

Code:

```
class Payment{
    void pay(){}
}
class CreditCardPayment extends Payment {
    void pay() {
        System.out.println("Paid by Credit Card");}
}
class PayPalPayment extends Payment {
    void pay() {
        System.out.println("Paid by Pay Pal");}
}
public class Three {
    public static void main(String[] args){
        Payment sakib=new CreditCardPayment();
        Payment adid=new PayPalPayment();
        sakib.pay();
        adid.pay();}
}
```

- 4. Design a class hierarchy for shapes. Create an abstract class Shape with a method draw(). Create subclasses Circle and Rectangle that override draw(). Write a program to call draw() on a Shape reference pointing to different objects.**

Code:

```
abstract class shape{
    double h,w,r;
    shape(double a, double b, double c){
        this.h=a; this.w=b; this.r=c;}
    abstract void draw();
    abstract double perimeter();
    abstract double area();
}
class Circle extends shape {
    Circle(double a) {
        super(0, 0, a);}
    double perimeter(){
        return 2*3.14*r;}
    double area(){
        return 3.14*r*r;}
    void draw() {
        System.out.println("Circle"+"\\n"+"Radius: "+r+"\\n"+"Perimeter: "+perimeter()+"\\n"+"Area: "+area()+"\\n");}
}
```

```

class Rectangle extends shape {
    Rectangle(double x, double y){
        super(x,y,0);
    }
    double perimeter(){
        return 2*(h+w);
    }
    double area(){
        return h*w;
    }
    void draw() {
        System.out.println("Rectangle"+"\\n"+"Height: "+h+"\\n"+"Width: "+w+"\\n"+"Perimeter: "+perimeter()+"\\n"+"Area: "+area()+"\\n");
    }
}

public class Four {
    public static void main(String[] args){
        Circle sakib=new Circle(3.5);
        Rectangle adid=new Rectangle(2,4);
        sakib.draw();
        adid.draw();
    }
}

```

- 5. In a company, different employees have different salary structures. Create a class Employee with a method calculateSalary(). Subclass FullTimeEmployee and PartTimeEmployee should override calculateSalary(). Demonstrate dynamic method dispatch in the main() method using an Employee reference.**

Code:

```

class Employee1 {
    String n, i;
    double s;
    Employee1(String a, String b, double c) {
        this.n = b; this.i = a; this.s = c;
    }
    double calculateSalary() {
        return 0;
    }
    void show() {}
}

class FullTimeEmployee1 extends Employee1 {
    FullTimeEmployee1(String a,String b, double c){
        super(a,b,c);
    }
    double calculateSalary() {
        return s*0.1+s;
    }
    void show() {
        System.out.println("ID: "+i+"\\n"+"Name: "+n+"\\n"+"salary per month: "+s+"\\n"+"Total salary: "+calculateSalary()+"\\n");
    }
}

class PartTimeEmployee1 extends Employee1 {
    PartTimeEmployee1(String a,String b, double c){
        super(a,b,c);
    }
    double calculateSalary() {
        return s*0.05+s;
    }
    void show() {

```

```

        System.out.println("ID: "+i+"\n"+"Name: "+n+"\n"+"salary per
month: "+s+"\n"+"Total salary: "+calculateSalary()+"\n");}
    }
    public class Five {
        public static void main(String[] args){
            Employee1 sakib=new FullTimeEmployee1("101","Sakib",12000);
            Employee1 adid=new PartTimeEmployee1("102","Adid",12000);
            sakib.show();
            adid.show();}
    }

```

- 6. An abstract class called Marks is needed to calculate the percentage of marks earned by students A in three subjects (with each subject out of 100) and student B in four subjects (with each subject out of 100). This class must contain the abstract method “getPercentage,” which two other classes, “A” and “B,” will inherit. The method “getPercentage,” which provides the percentage of students, is shared by classes “A” and “B.” The constructor of class ‘A’ will accept the marks obtained in three subjects as its parameters and the constructor of class ‘B’ will accept the marks obtained in four subjects as its parameters. To test the implementation, objects for both the classes need to be created and the percentage of marks for each student should be printed.**

Code:

```

abstract class Marks {
    int m1, m2, m3, m4;
    Marks(int m1, int m2, int m3, int m4) {
        this.m1 = m1; this.m2 = m2; this.m3 = m3; this.m4 = m4;}
    abstract double getPercentage();}
class A extends Marks {
    A(int x, int y, int z) {
        super(x,y,z,0);}
    double getPercentage() {
        return (m1 + m2 + m3) / 3;}
}
class B extends Marks {
    B(int w, int x, int y, int z) {
        super(w,x,y,z);}
    double getPercentage() {
        return (m1 + m2 + m3 + m4) / 4.0;}
}
public class Six {
    public static void main(String[] args) {
        Marks A = new A(85, 90, 80);
        Marks B = new B(75, 85, 80, 90);
        System.out.println("Percentage of Student A: " + A.getPercentage()
+ "%");
    }
}

```

```
        System.out.println("Percentage of Student B: " + B.getPercentage()  
+ "%");}  
}
```